Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#09)

Implementation of a Register File using ModelSim

Student Name: _____*Syed Sikandar Abbas*__ Roll No: __*i246553*__ Section: *CE-B*

Date performed: ____*October 21*, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:

Verilog code for 4x4 register file.

Code:

```
module decoder_2_to4 (output reg [3:0] dec_out, input [1:0] wr, input write_enable);
	always @(*) begin
		if (write_enable) begin
			case (wr)
				2'b00: dec_out = 4'b0001;
				2'b01: dec_out = 4'b0010;
				2'b10: dec_out = 4'b0100;
				2'b11: dec_out = 4'b1000;
				default: dec_out = 4'b0000;
			endcase
		end else begin
			dec_out = 4'b0000;
		end
	end
endmodule

module mux4_1 (output reg [3:0] out, input [3:0] in, in1, in2, in3, input [1:0] se1);
	always @(*) begin
		if (se1 == 2'b00)
			out = in;
		else if (se1 == 2'b01)
			out = in1;
		else if (se1 == 2'b10)
			out = in2;
		else if (se1 == 2'b11)
			out = in3;
		else
			out = 4'b0000; // optional, for safety
	end
endmodule

module register_4bit (output reg [3:0] q, input [3:0] d, input load, input clk);
	initial begin
		q <= 4'b0000;
	end
	always @(posedge clk) begin
		if (load)
			q <= d;
	end
endmodule

module register_file_4x4 (data_out1, data_out2, data_in, wr, write_enable, rd1, rd2, clk);
	// output ports
	output [3:0] data_out1;
	output [3:0] data_out2;

	// input ports
	input [3:0] data_in;
	input [1:0] wr;
```

```
        input write_enable;
        input [1:0] rd1, rd2;
        input clk;

        // Internal wires
        wire [3:0] dec_out;
        wire [3:0] q0, q1, q2, q3;

        // Instantiate decoder
        decoder_2_to4 decoder (dec_out, wr, write_enable);

        // Instantiate four 4-bit registers
        register_4bit reg0 (q0, data_in, dec_out [0], clk);
        register_4bit reg1 (q1, data_in, dec_out [1], clk);
        register_4bit reg2 (q2, data_in, dec_out [2], clk);
        register_4bit reg3(q3, data_in, dec_out [3], clk);

        // Instantiate two 4-to-1 multiplexers for reading
        mux4_1 mux1 (data_out1, q0, q1, q2, q3, rd1);
        mux4_1 mux2 (data_out2, q0, q1, q2, q3, rd2);
endmodule
```

Task 2:

Description:

Verilog Code & Testbench for 4x4 Register file implemented in Task 1.

Code:

```
module decoder_2_to4 (output reg [3:0] dec_out, input [1:0] wr, input write_enable);
	always @(*) begin
		if (write_enable) begin
			case (wr)
				2'b00: dec_out = 4'b0001;
				2'b01: dec_out = 4'b0010;
				2'b10: dec_out = 4'b0100;
				2'b11: dec_out = 4'b1000;
				default: dec_out = 4'b0000;
			endcase
		end else begin
			dec_out = 4'b0000;
		end
	end
endmodule

module mux4_1 (output reg [3:0] out, input [3:0] in, in1, in2, in3, input [1:0] se1);
	always @(*) begin
		if (se1 == 2'b00)
			out = in;
		else if (se1 == 2'b01)
			out = in1;
		else if (se1 == 2'b10)
			out = in2;
		else if (se1 == 2'b11)
			out = in3;
		else
			out = 4'b0000; // optional, for safety
	end
endmodule

module register_4bit (output reg [3:0] q, input [3:0] d, input load, input clk);
	initial begin
		q <= 4'b0000;
	end
	always @(posedge clk) begin
		if (load)
			q <= d;
	end
endmodule

module register_file_4x4 (data_out1, data_out2, data_in, wr, write_enable, rd1, rd2, clk);
	// output ports
	output [3:0] data_out1;
	output [3:0] data_out2;

	// input ports
	input [3:0] data_in;
	input [1:0] wr;
	input write_enable;
```

```
        input [1:0] rd1, rd2;
        input clk;

        // Internal wires
        wire [3:0] dec_out;
        wire [3:0] q0, q1, q2, q3;

        // Instantiate decoder
        decoder_2_to4 decoder (dec_out, wr, write_enable);

        // Instantiate four 4-bit registers
        register_4bit reg0 (q0, data_in, dec_out [0], clk);
        register_4bit reg1 (q1, data_in, dec_out [1], clk);
        register_4bit reg2 (q2, data_in, dec_out [2], clk);
        register_4bit reg3(q3, data_in, dec_out [3], clk);

        // Instantiate two 4-to-1 multiplexers for reading
        mux4_1 mux1 (data_out1, q0, q1, q2, q3, rd1);
        mux4_1 mux2 (data_out2, q0, q1, q2, q3, rd2);
endmodule

module tb_4bit_RFile();
        reg [3:0] d_in;
        reg [1:0] wr;
        reg write_en;
        reg [1:0] rd1, rd2;
        reg clk;
        wire [3:0] out1;
        wire [3:0] out2;

        register_file_4x4 RFile(
                .data_out1(out1),
                .data_out2(out2),
                .data_in(d_in),
                .wr(wr),
                .write_enable(write_en),
                .rd1(rd1),
                .rd2(rd2),
                .clk(clk)
        );

        always begin
                #5 clk = ~clk;
        end

        initial begin
                clk = 0;
                write_en = 0;
                d_in = 0'b0;
                wr = 2'b0;
                rd1 = 2'b00;
                rd2 = 2'b01;

                #10
                write_en = 1;

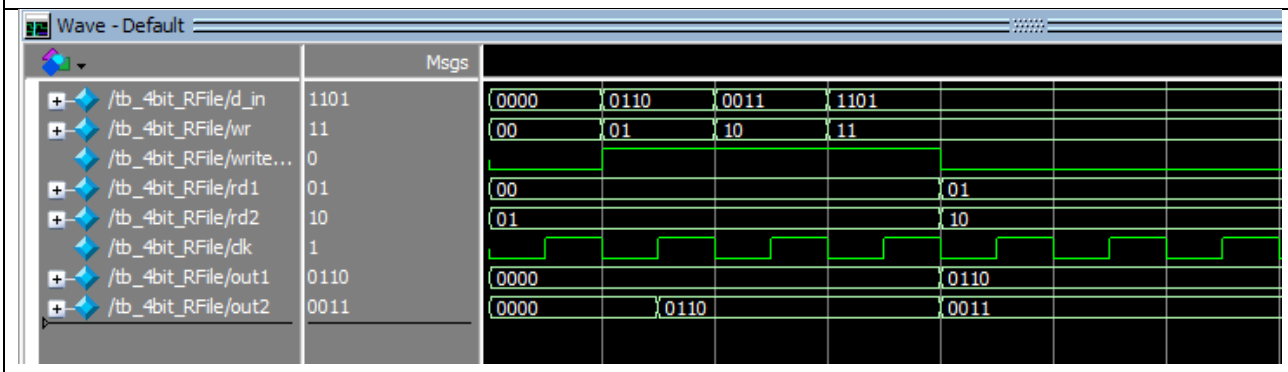
                // i) ------ R1 <- 6 & R2 <- 3
                d_in = 4'b0110;
                wr = 2'b01;
                #10
                d_in = 4'b0011;
```

```
                wr = 2'b10;
                #10

                // ii) ------ R3 <- 13
                d_in = 4'b1101;
                wr = 2'b11;
                #10

                // iii) ----- out1 <- R1 & out2 <- R2
                write_en = 0;
                rd1 = 2'b01;
                rd2 = 2'b10;
        end
endmodule
```

Output:



Task 3:

Description:

A 32-bit Register File for a RISC-V (RV32) processor architecture was designed, implemented and verified using Verilog.

Code:

```
module decoder_5_to_32 (
    output reg [31:0] dec_out,
    input [4:0] wr,
    input write_enable
);
    always @(*) begin
        if (write_enable && (wr != 5'b00000)) begin
            dec_out = 32'b1 << wr;
        end else begin
            dec_out = 32'h00000000;
        end
    end
endmodule

module register_32bit (
    output reg [31:0] q,
    input [31:0] d,
    input load,
    input clk,
    input rst
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 32'h00000000;
        else if (load)
            q <= d;
    end
endmodule

module mux32_1 (
    output reg [31:0] out,
    input [31:0] in0, in1, in2, in3, in4, in5, in6, in7,
                 in8, in9, in10, in11, in12, in13, in14, in15,
                 in16, in17, in18, in19, in20, in21, in22, in23,
                 in24, in25, in26, in27, in28, in29, in30, in31,
    input [4:0] sel
);
    always @(*) begin
        case (sel)
            5'd0:  out = 32'h00000000; //x0 is hardwired to 0
            5'd1:  out = in1;
            5'd2:  out = in2;
            5'd3:  out = in3;
            5'd4:  out = in4;
            5'd5:  out = in5;
            5'd6:  out = in6;
            5'd7:  out = in7;
            5'd8:  out = in8;
            5'd9:  out = in9;
            5'd10: out = in10;
```

```
            5'd11: out = in11;
            5'd12: out = in12;
            5'd13: out = in13;
            5'd14: out = in14;
            5'd15: out = in15;
            5'd16: out = in16;
            5'd17: out = in17;
            5'd18: out = in18;
            5'd19: out = in19;
            5'd20: out = in20;
            5'd21: out = in21;
            5'd22: out = in22;
            5'd23: out = in23;
            5'd24: out = in24;
            5'd25: out = in25;
            5'd26: out = in26;
            5'd27: out = in27;
            5'd28: out = in28;
            5'd29: out = in29;
            5'd30: out = in30;
            5'd31: out = in31;
            default: out = 32'h00000000;
        endcase
    end
endmodule

module register_file_32x32 (
    output [31:0] rdata1,
    output [31:0] rdata2,
    input [31:0] wdata,
    input [4:0] rd,
    input [4:0] rs1,
    input [4:0] rs2,
    input reg_write,
    input clk,
    input rst
);

    wire [31:0] dec_out;
    wire [31:0] q0, q1, q2, q3, q4, q5, q6, q7,
                q8, q9, q10, q11, q12, q13, q14, q15,
                q16, q17, q18, q19, q20, q21, q22, q23,
                q24, q25, q26, q27, q28, q29, q30, q31;

    assign q0 = 32'h00000000;

    decoder_5_to_32 decoder (dec_out, rd, reg_write);

    register_32bit reg1  (q1,  wdata, dec_out[1],  clk, rst);
    register_32bit reg2  (q2,  wdata, dec_out[2],  clk, rst);
    register_32bit reg3  (q3,  wdata, dec_out[3],  clk, rst);
    register_32bit reg4  (q4,  wdata, dec_out[4],  clk, rst);
    register_32bit reg5  (q5,  wdata, dec_out[5],  clk, rst);
    register_32bit reg6  (q6,  wdata, dec_out[6],  clk, rst);
    register_32bit reg7  (q7,  wdata, dec_out[7],  clk, rst);
    register_32bit reg8  (q8,  wdata, dec_out[8],  clk, rst);
    register_32bit reg9  (q9,  wdata, dec_out[9],  clk, rst);
    register_32bit reg10 (q10, wdata, dec_out[10], clk, rst);
    register_32bit reg11 (q11, wdata, dec_out[11], clk, rst);
    register_32bit reg12 (q12, wdata, dec_out[12], clk, rst);
    register_32bit reg13 (q13, wdata, dec_out[13], clk, rst);
```

```
    register_32bit reg14 (q14, wdata, dec_out[14], clk, rst);
    register_32bit reg15 (q15, wdata, dec_out[15], clk, rst);
    register_32bit reg16 (q16, wdata, dec_out[16], clk, rst);
    register_32bit reg17 (q17, wdata, dec_out[17], clk, rst);
    register_32bit reg18 (q18, wdata, dec_out[18], clk, rst);
    register_32bit reg19 (q19, wdata, dec_out[19], clk, rst);
    register_32bit reg20 (q20, wdata, dec_out[20], clk, rst);
    register_32bit reg21 (q21, wdata, dec_out[21], clk, rst);
    register_32bit reg22 (q22, wdata, dec_out[22], clk, rst);
    register_32bit reg23 (q23, wdata, dec_out[23], clk, rst);
    register_32bit reg24 (q24, wdata, dec_out[24], clk, rst);
    register_32bit reg25 (q25, wdata, dec_out[25], clk, rst);
    register_32bit reg26 (q26, wdata, dec_out[26], clk, rst);
    register_32bit reg27 (q27, wdata, dec_out[27], clk, rst);
    register_32bit reg28 (q28, wdata, dec_out[28], clk, rst);
    register_32bit reg29 (q29, wdata, dec_out[29], clk, rst);
    register_32bit reg30 (q30, wdata, dec_out[30], clk, rst);
    register_32bit reg31 (q31, wdata, dec_out[31], clk, rst);

    mux32_1 mux1 (rdata1, q0, q1, q2, q3, q4, q5, q6, q7,
                          q8, q9, q10, q11, q12, q13, q14, q15,
                          q16, q17, q18, q19, q20, q21, q22, q23,
                          q24, q25, q26, q27, q28, q29, q30, q31, rs1);

    mux32_1 mux2 (rdata2, q0, q1, q2, q3, q4, q5, q6, q7,
                          q8, q9, q10, q11, q12, q13, q14, q15,
                          q16, q17, q18, q19, q20, q21, q22, q23,
                          q24, q25, q26, q27, q28, q29, q30, q31, rs2);

endmodule

module tb_register_file_32x32();
       reg [31:0] d_in;
       reg [4:0] wr;
       reg write_en;
       reg [4:0] rd1, rd2;
       reg clk;
       reg rst;
       wire [31:0] out1;
       wire [31:0] out2;

       // Instantiate the 32x32 RISC-V Register File
       register_file_32x32 RFile(
              .rdata1(out1),
              .rdata2(out2),
              .wdata(d_in),
              .rd(wr),
              .rs1(rd1),
              .rs2(rd2),
              .reg_write(write_en),
              .clk(clk),
              .rst(rst)
       );

       always begin
              #5 clk = ~clk;
       end

       initial begin
              clk = 0;
              rst = 1;
```

```
                write_en = 0;
                d_in = 32'b0;
                wr = 5'b00000;
                rd1 = 5'b00000;
                rd2 = 5'b00000;

                #10;
                rst = 0;
                #10;
                write_en = 1;

                d_in = 32'hABCDEFAB;
                wr = 5'b00000;
                #10;

                d_in = 32'd6;
                wr = 5'b00001; // register x1
                #10;

                d_in = 32'd3;
                wr = 5'b00010; // register x2
                #10;

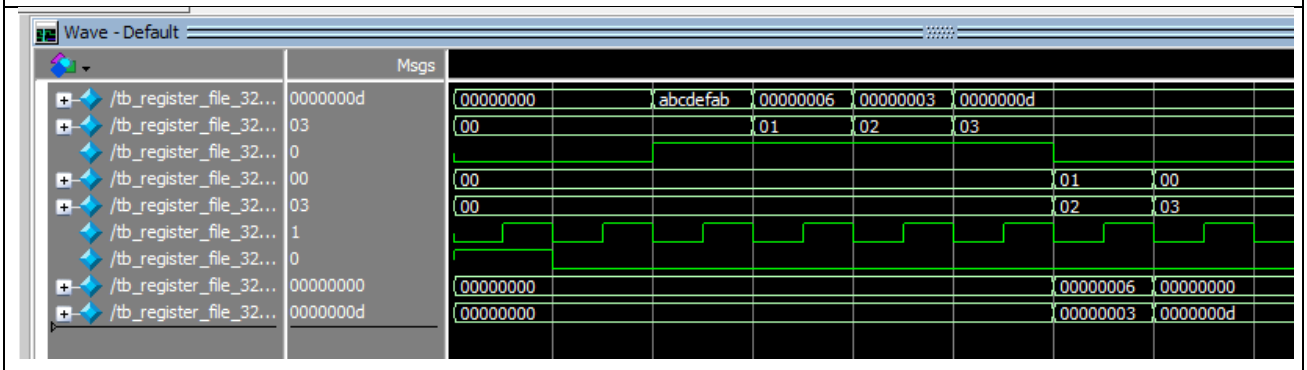
                d_in = 32'd13;
                wr = 5'b00011; // register x3
                #10;

                write_en = 0;
                rd1 = 5'b00001; // select x1 for read port 1
                rd2 = 5'b00010; // select x2 for read port 2
                #10;

                rd1 = 5'b00000; // select x0 (should be 0)
                rd2 = 5'b00011; // select x3 (should be 13)
                #10;

        end
endmodule
```

Output:



Task 4:

Description:
Simulated a 32-bit Register in Logisim.
Circuit:

